Лекция 3. Архитектуры систем управления РРТС

Центр, соседи и иерархия: кто принимает решения в группе роботов и чем за это платит

90 минут · две части по 45

Курс «Управление распределёнными робототехническими системами»

План лекции

1. Архитектура PPTC: определение, роль, цикл «восприятие — вычисление — действие»
2. Формальная модель S = ⟨A, C, G, П⟩ и классификация архитектур
3. Централизованная архитектура: звезда, глобальный оптимум, узкое место
4. Децентрализованная архитектура: соседи, консенсус, живучесть
5. Гибридные и иерархические архитектуры
6. Сравнительный анализ и численный пример нагрузки на связь
7. Алгоритмы управления в разных архитектурах
8. ROS и ROS 2 как архитектурно нейтральный каркас
9. Практические примеры: склад, рой БПЛА, автономный транспорт
10. Проблемы реализации и направления их решения

Синие — первая часть · янтарные — вторая часть

Часть 1. Два полюса: центр и соседи

От цикла «восприятие — действие» отдельного агента к централизованной звезде и децентрализованной сети соседей.

1 Что такое архитектура системы управления

– Организационная структура: состав компонентов и распределение права решать

– Направления и интенсивность информационных потоков между агентами

– Способы координации ради общесистемной цели, а не локальных целей

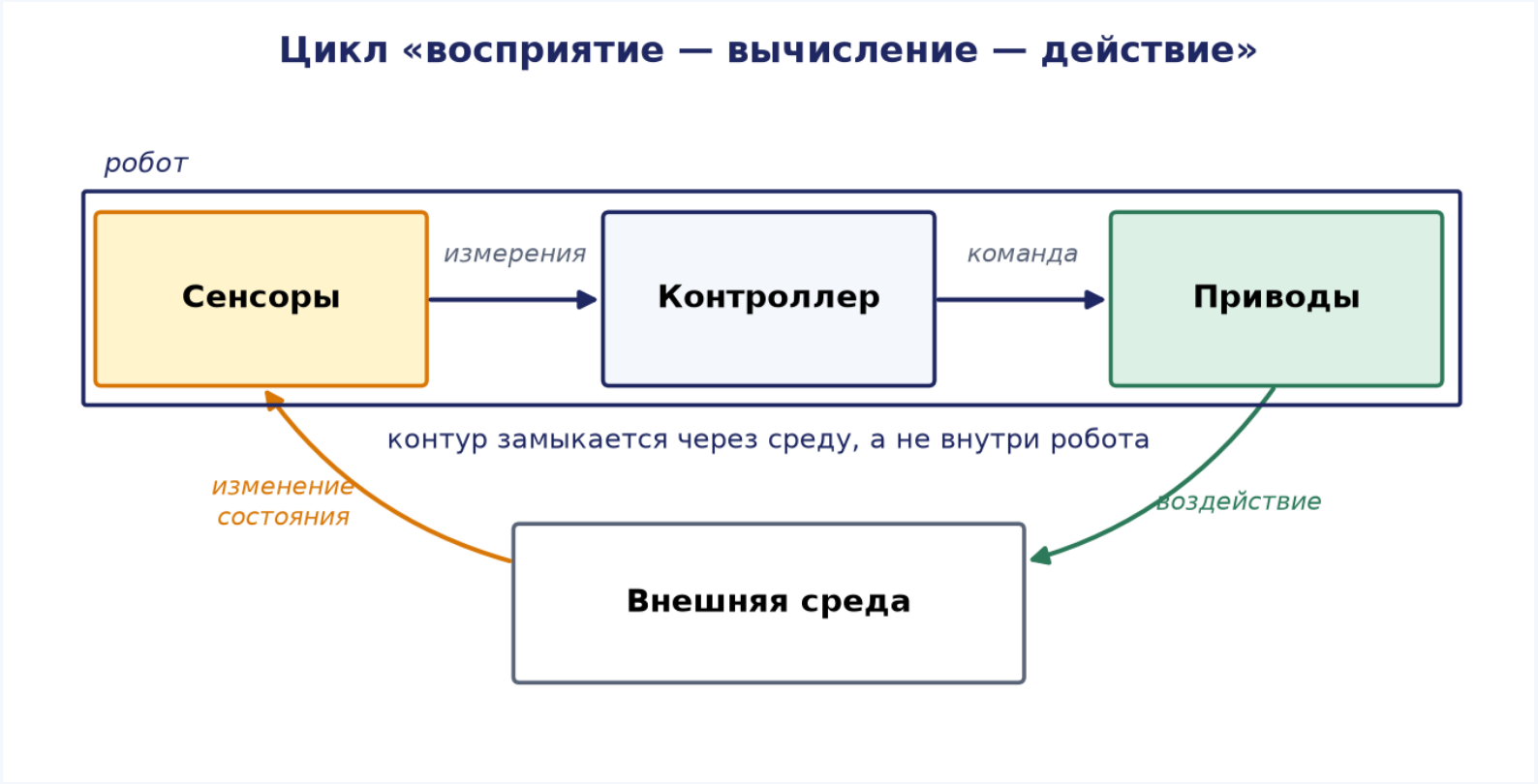
– Один робот описывается кинематикой и сенсорикой, группа — архитектурой

– Ключевой вопрос: кто, что и на основании какой информации решает

Ключевая идея

Архитектура — это ответ на вопрос «кто решает». Два одинаковых по железу флота роботов с разной архитектурой ведут себя как разные системы.

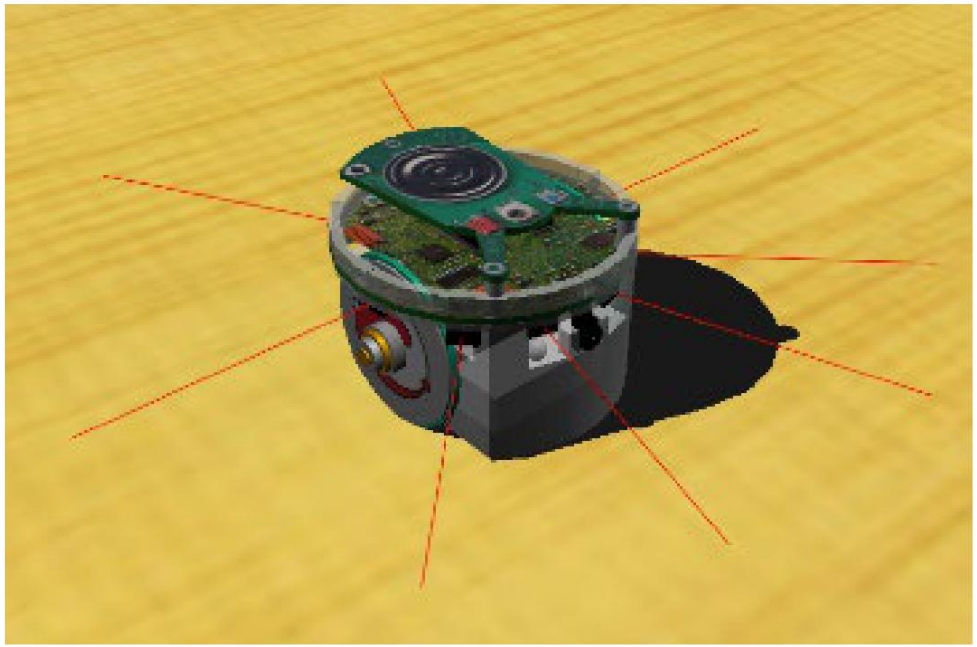
1 Цикл «восприятие — вычисление — действие»



Восприятие (сенсоры) → вычисление (контроллер) → действие (приводы); контур замыкается обратной связью через среду. Это базовая единица поведения робота и «кирпич», из которого собирается РРТС.

1 Робот e-puck: реальный и симулированный



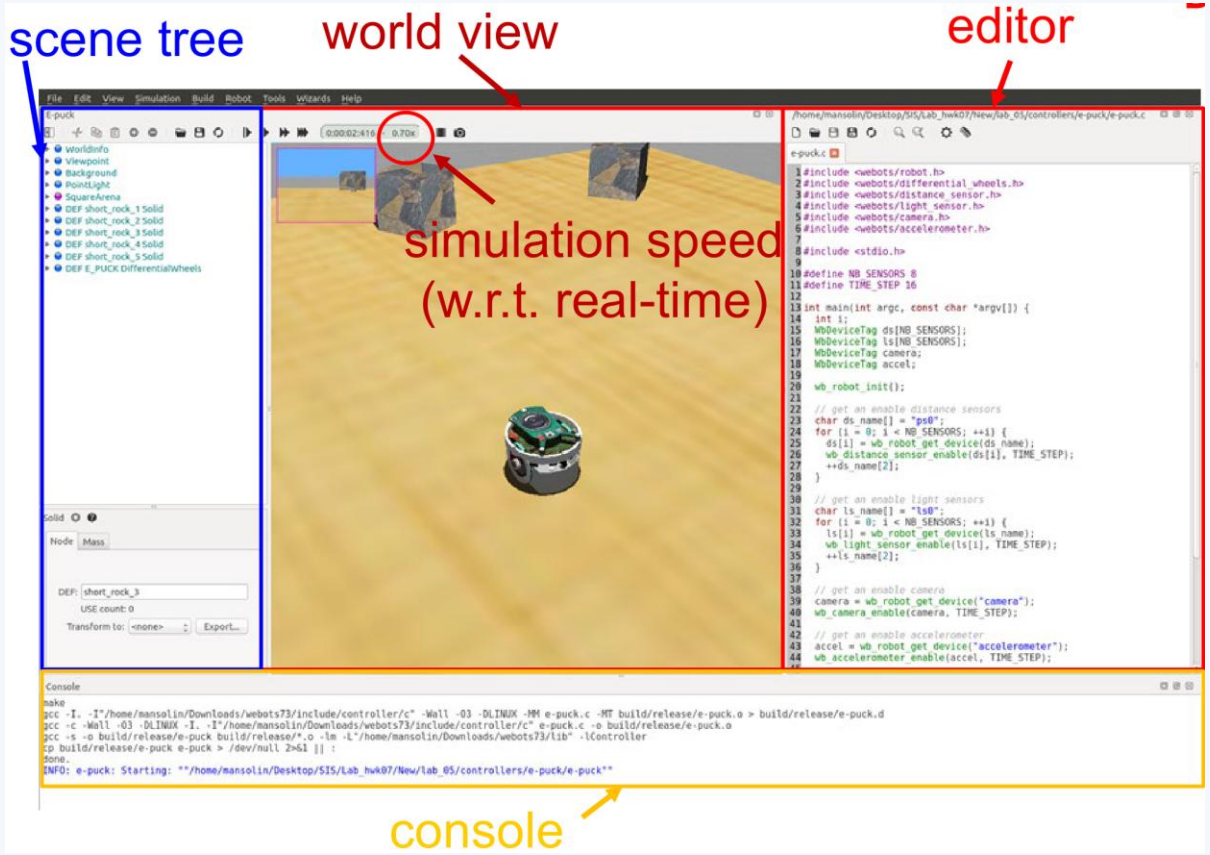


Simulated e-puck (Webots)

e-puck: цилиндр Ø 70 мм, процессор dsPIC, два шаговых мотора, кольцо LED, камера, 8 ИК-датчиков близости, 3D-акселерометр, Bluetooth. Слева — реальный робот, справа — его модель в Webots.

Адаптировано из EPFL DIS, W3

1 Интерфейс симулятора Webots



GUI Webots: вид мира, дерево сцены, редактор кода, консоль и индикатор скорости симуляции относительно реального времени.

Адаптировано из EPFL DIS, W3

2 Формальная модель PPTC

$$S = \langle A, C, G, \Pi \rangle$$

– A = {a_1, …, a_n} — множество агентов (роботов)

– C — множество управляющих узлов: пусто, совпадает с A или отдельный уровень

– G = (V, E) — коммуникационный граф: вершины — узлы, рёбра — каналы связи

– П — протоколы и алгоритмы координации, определённые на графе G

Тип архитектуры задаётся структурой графа G и тем, как право решать распределено между A и C.

2 Что определяет выбор архитектуры

– Эффективность: оптимальность траекторий, распределения задач, ресурсов

– Масштабируемость: сохранит ли система работоспособность при росте n

– Отказоустойчивость: что будет при потере агента, контроллера или канала

– Сложность реализации: трудоёмкость алгоритмов, стоимость связи и вычислителей

– Решение принимается в начале проекта и определяет весь его жизненный цикл

“

Архитектура — не бинарный выбор «центр или не центр», а точка в непрерывном спектре степеней централизации.

Главный тезис главы 3

2 Три типа архитектур: карта местности

Тип	Кто решает	Граф G
Централизованная	Единственный узел c_o	Звезда: агент ↔ центр
Децентрализованная	Каждый агент сам, C = ∅	Произвольный связный граф соседей
Гибридная	Центр — стратегия, борт — тактика	Иерархия: центр ↔ координаторы ↔ агенты

Между строками таблицы — непрерывный переход, а не три изолированные точки.

3 Централизованная архитектура: структура

– Все агенты шлют в центр c_0 вектор состояния x_i: позиция, скорость, заряд, статус

– Центр видит полную картину $x = (x_1, \ldots, x_n)$ и рассылает команды $u_1, \ldots, u_n$

– Топология — звезда: $E = \{(a_i, c_0)\}$, рёбер между агентами нет

– Агент — исполнитель: он не знает о существовании соседей

– Центр решает единую задачу над полным состоянием — отсюда глобальный оптимум

Кейс

Классика жанра — складской комплекс: центральная система видит всю карту, все заказы и все роботы, назначает задачи и резервирует ячейки пространства-времени.

3 Задача центрального контроллера

$$\min \sum_{i=1}^{n} f_i(x_i, u_i) \text{ при } g(x, u) \leq 0$$

– $f_i(x_i, u_i)$ — функция стоимости i-го агента: пробег, энергия, время

– $g(x, u) \leq 0$ — связи между агентами: бесстолкновительность, узкие проходы, ресурсы

– Минимум берётся сразу по всему вектору $(u_1, \ldots, u_n)$

– Именно совместность задачи и даёт глобально оптимальное решение

Всё преимущество централизации спрятано в одном символе — общей сумме по всем агентам сразу.

3 Цена централизации: связь и вычисления

– Связь: 2n сообщений за такт — линейно, O(n), но всё через один узел

– Центр — bottleneck: его канал и вычислитель ограничивают всю систему

– Попарные ограничения бесстолкновительности: $n(n-1)/2 = O(n^2)$

– Оптимальное назначение задач венгерским алгоритмом — $O(n^3)$

– Совместное планирование движения в общем случае PSPACE-трудно

3 Численный пример: склад на 400 роботов

12,8 Мбит/с

поток через центральный узел при n = 400, такте 0,1 с

128 Мбит/с

тот же поток при n = 4000 — через одну вершину

×1000

рост вычислительной задачи назначения при $O(n^3)$

- Сообщение состояния и команда — по 200 байт, 10 тактов в секунду
- Трафик реализуем, но вся нагрузка сходится в одну точку

? Как думаете?

Централизованный склад растёт с 400 до 4000 роботов. Что упрётся в предел первым?

- **A** Пропускная способность канала до центра
- **B** Вычислительное время планирования и назначения
- **C** Память центрального сервера
- **D** Ёмкость аккумуляторов роботов

? Как думаете?

Централизованный склад растёт с 400 до 4000 роботов. Что упрётся в предел первым?

- **A** Пропускная способность канала до центра
- **B** Вычислительное время планирования и назначения
- **C** Память центрального сервера
- **D** Ёмкость аккумуляторов роботов

ОТВЕТ

Вычисления. Трафик растёт линейно (12,8 → 128 Мбит/с), а задача назначения при $O(n^3)$ тяжелеет в 1000 раз — счёт упирается в такт раньше, чем канал в предел.

3 Единая точка отказа

– Отказ центра останавливает систему целиком, независимо от состояния агентов

– Вероятность работоспособности системы не превышает p_0 — надёжности центра

– Резервирование: m независимых контроллеров снижают отказ до $(1 - p_0)^m$

– Плата — стоимость и новая задача: переключение на резерв без потери состояния

– Выбор лидера и репликация состояния — отдельная нетривиальная проблема

Факт

Резервирование не отменяет проблему, а переносит её: вместо «надёжен ли центр» появляется вопрос «надёжен ли механизм переключения на резерв».

4 Децентрализованная архитектура: структура

– $C = \emptyset$: центра нет, каждый агент решает сам

– Основа решения: своё состояние x_i, локальные наблюдения, данные соседей N_i

– Топология задаётся физически: $(a_i, a_j) \in E \Leftrightarrow \|q_i - q_j\| \le R$

– Локальная задача: $u_i = \arg\min [f_i(x_i, u_i) + \Sigma_{j \in N_i} g_{ij}(x_i, x_j, u_i, u_j)]$

– Глобальное поведение не программируется, а возникает эмерджентно

Ключевая идея

Граф связи здесь не проектируется на бумаге, а определяется геометрией: кто рядом — тот и сосед. Топология меняется по ходу движения сама.

4 Протокол усредняющего консенсуса

$$x_i(k+1) = x_i(k) + \varepsilon \cdot \sum_{j \in N_i} (x_j(k) - x_i(k))$$

– Агент подтягивает своё значение к значениям соседей — и больше ничего

– При связном G и $0 < \varepsilon < 1/d_max$ все x_i сходятся к среднему начальных значений

– Скорость сходимости определяет алгебраическая связность $\lambda_2(L)$ лапласиана

– Плотнее граф — больше λ_2 — быстрее согласование, но дороже радиоканал

Одна строчка кода на борту — и группа без центра приходит к общему числу.

4 Масштабируемость и её цена

– Полносвязная сеть даёт $O(n^2)$ сообщений — хуже, чем звезда

– Но геометрия ограничивает число соседей: d_max — константа, не зависящая от n

– Итог: нагрузка на узел O(d_max) = O(1), суммарная — O(n) и распределена

– Тысячный агент нагружает систему не больше, чем десятый

– Плата: информация идёт через сеть за O(D(G)) тактов — диаметр графа

Пример

400 роботов, по 6 соседей: нагрузка на узел 192 кбит/с — константа при любом n. Диаметр графа ≈ 20 прыжков, то есть глобальное согласование займёт около 2 секунд.

4 Центр против соседей: расстановка сил

Централизованная

- Глобальный оптимум решений
- Простая и прозрачная логика
- Удобны мониторинг и сертификация
- Нагрузка на центр O(n)
- Единая точка отказа

Децентрализованная

- Живучесть и плавная деградация
- Нагрузка на узел O(1)
- Мгновенная локальная реакция
- Субоптимальность поведения
- Трудно верифицировать и наблюдать

4 Живучесть: деградация и византийцы

– Отказ агента = удаление вершины; связный остаток продолжает работать

– Плавная деградация: производительность падает пропорционально, а не в ноль

– Мера структурной живучести — вершинная связность κ(G)

– Система переживает отказ любых κ(G) − 1 агентов; на практике держат κ ≥ 2–3

– Опаснее отказов — ложные данные: f византийцев требуют не менее 3f + 1 участников

Факт

Робастный протокол W-MSR: агент отбрасывает f наибольших и f наименьших значений соседей и усредняет только оставшиеся — простая идея, спасающая консенсус от «отравления».

Конец первой части

Разобрали два полюса: центр даёт оптимум и одну точку отказа, рой даёт живучесть и субоптимальность. После перерыва — как их соединить, чем платят гибриды и как всё это выглядит в ROS 2 и в реальных проектах.

Часть 2. Гибрид, платформы и практика

Иерархия уровней, кластеризация, сводное сравнение, алгоритмы, ROS 2 и три реальных кейса.

5 Гибрид: разделение по масштабам

– Медленные глобальные решения — центру: задачи, маршруты, ресурсы

– Быстрые локальные решения — борту: объезд, стабилизация, расхождение

– Критерий разделения — характерное время, а не важность решения

– Верхний уровень оперирует агрегированным состоянием и обновляется редко

– Нижний наследует от роя постоянную нагрузку и мгновенную реакцию

Ключевая идея

Гибрид разрезает задачу не по агентам, а по времени: что можно решать раз в минуту — наверх, что нужно решать каждые 10 мс — вниз.

5 Три уровня иерархии

– Стратегический (центр): минуты—часы, глобальная оптимизация, связь с ERP

– Тактический (координаторы групп): секунды—минуты, координация внутри кластера

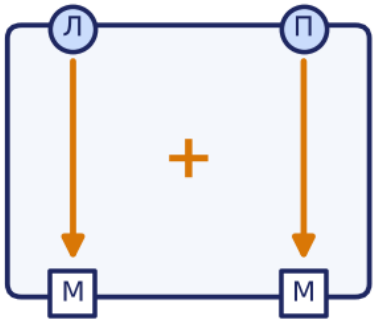
– Исполнительный (борт): миллисекунды—секунды, приводы, торможение, объезд

– Каждый уровень передаёт вниз лишь необходимую часть своего решения

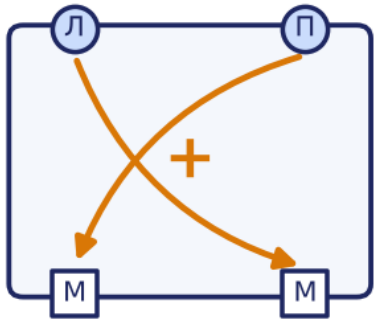
– Рефлекторный уровень наследует идеи классических реактивных архитектур

5 Транспортные средства Брайтенберга

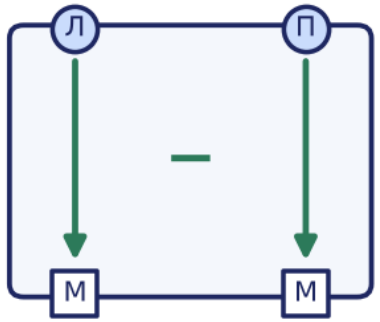
Л, П — левый и правый датчик; М — мотор; знак и величина коэффициента задают поведение



прямые связи, возбуждение:
робот уходит от источника



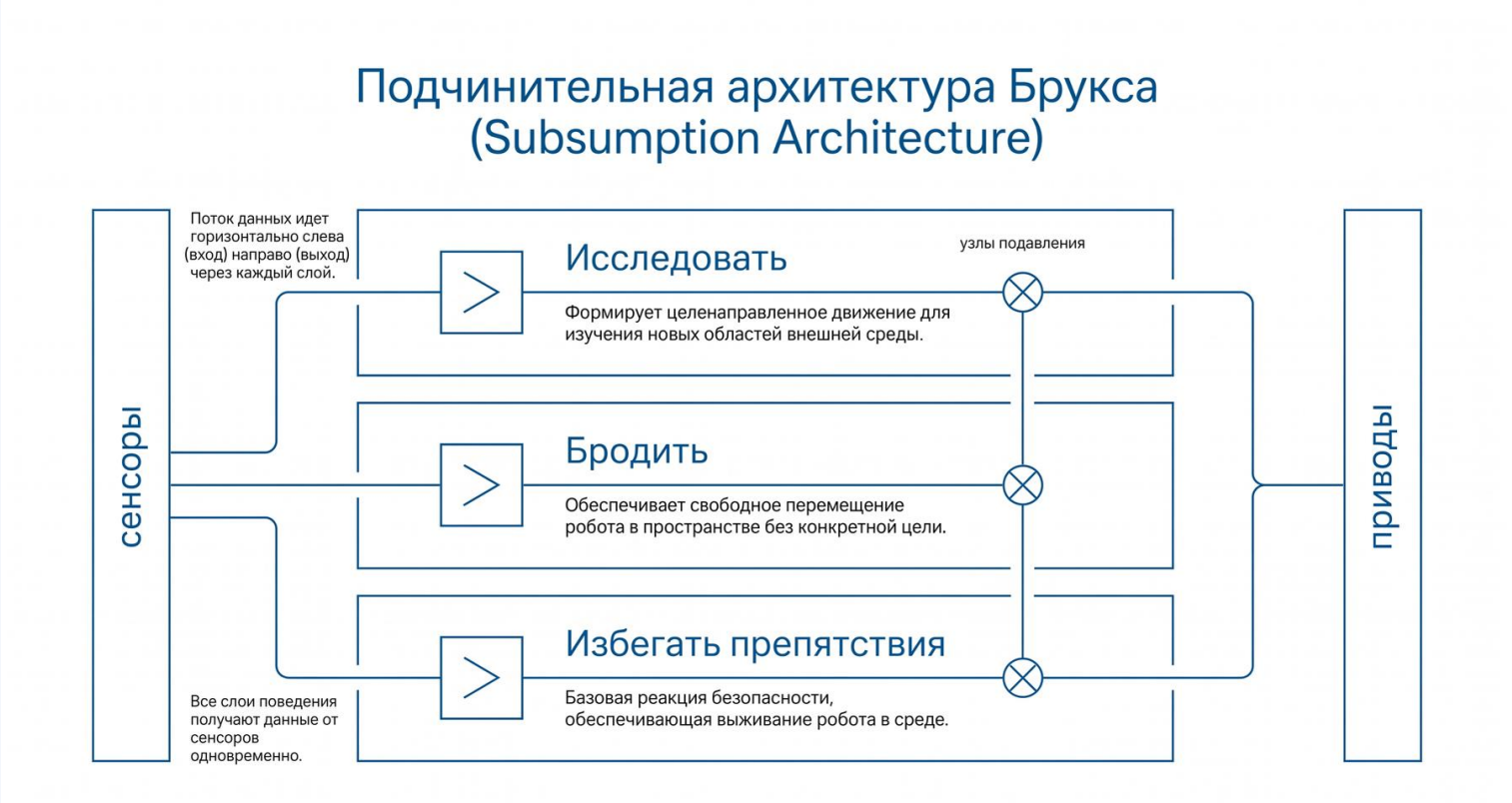
перекрёстные связи, возбуждение:
робот идёт на источник



прямые связи, торможение:
робот замирает у источника

Прямые связи «датчик → мотор»: «+» — возбуждение, «−» — торможение. Перекрёстные и прямые схемы соединения дают поведение «страх», «агрессия», избегание препятствий.

5 Субсумпционная архитектура Брукса



Слои поведения работают параллельно; верхний слой «избегание препятствий» подавляет (suppress) и замещает выход нижнего слоя «блуждание» — в отличие от последовательной схемы Sense–Model–Plan–Act.

5 Двухуровневая задача оптимизации

$$\text{верх} \;: \min_{U} \sum_{i} F_i(\bar{x}_i, U) \rightarrow \;\; \text{низ} \;\; : u_i = \arg\min f_i(x_i, u_i; U_i)$$

– U — глобальные решения: назначения задач, опорные маршруты, целевые зоны

– $\bar{x}_i$ — агрегированное, огрублённое состояние агента или группы

– U_i приходит вниз как параметр, а не как жёсткая команда

– Внизу к нему добавляется сумма по соседям $\Sigma_{j \in N_i} g_{ij}$ — локальное расхождение

Верх оперирует агрегатами и обновляется редко; низ работает непрерывно и по соседям.

5 Многоуровневая структура системы управления PPTC



Направления передачи информации в иерархии

Направление	Тип сообщения	Описание
Сверху вниз	Команды	Управляющие сигналы от оператора к исполнителям
Снизу вверх	Донесения	Данные о состоянии роботов и среды для анализа

Оператор и управляющий центр (стратегический уровень) → блоки декомпозиции задач и формирования траекторий (тактический уровень) → сенсоры, приводы и навигация отдельных роботов (исполнительный уровень) → внешняя среда.

5 Федеративная организация управления



Глобальный координатор согласует работу групп; внутри каждой группы агенты взаимодействуют напрямую. Такая структура ограничивает межгрупповую коммуникацию и локализует разрешение конфликтов.

5 Кластеризация: арифметика иерархии

– n агентов делим на m групп по n/m; в каждой — координатор

– Нагрузка на центр падает с O(n) до O(m), на координатора — O(n/m)

– При m = √n обе величины балансируются на уровне O(√n)

– L уровней с ветвлением b обслуживают $n = b^L$ агентов при нагрузке O(b) на узел

– Глубина распространения команды — O(log_b n), логарифм вместо линии

Ключевая идея

Иерархия меняет линейную по n нагрузку на самый загруженный узел на величину порядка √n или даже log n — это и есть инженерный ответ на проблему масштаба.

? Проверим арифметику

n = 900 роботов. Сколько кластеров выбрать, чтобы сбалансировать нагрузку центра и координаторов?

- **A** 9 кластеров по 100 роботов
- **B** 30 кластеров по 30 роботов
- **C** 100 кластеров по 9 роботов
- **D** Без разницы — суммарный трафик один и тот же

? Проверим арифметику

n = 900 роботов. Сколько кластеров выбрать, чтобы сбалансировать нагрузку центра и координаторов?

- **A** 9 кластеров по 100 роботов
- **B** 30 кластеров по 30 роботов
- **C** 100 кластеров по 9 роботов
- **D** Без разницы — суммарный трафик один и тот же

ОТВЕТ

30 кластеров по 30: при m = √n = 30 нагрузка центра O(m) и нагрузка координатора O(n/m) равны, обе порядка 30 соединений.

6 Сводное сравнение архитектур

Критерий	Централизованная	Децентрализованная	Гибридная
Нагрузка на худший узел	O(n) — центр	O(1) — константа	O(√n) или O(b)
Качество решений	Глобальный оптимум	Локальный, эмерджентный	Близко к глобальному
Единая точка отказа	Есть	Нет	Локализована уровнем
Реакция на локальное событие	Цикл через центр	Минимальная, на борту	Минимальная, на борту
Глобальное согласование	Один такт центра	O(D(G)) тактов	O(log n) уровней
Сложность разработки	Низкая — средняя	Высокая	Наивысшая
Типичный масштаб	10–10^3 агентов	10^2–10^5 агентов	10^2–10^4 агентов

Гибрид выигрывает почти по всем строкам, кроме одной — сложности разработки.

6 Три варианта одного склада: n = 400

12,8 Мбит/с

централизованная: всё через центральный узел

192 кбит/с

децентрализованная: на узел, не зависит от n

640 кбит/с

гибридная: и на центр, и на координатора при m = 20

– Децентрализованная даёт 38,4 Мбит/с суммарно, но распределённо по пространству

– При росте до n = 4000 гибридная нагрузка на узел вырастет лишь до ≈ 2 Мбит/с

6 Критерии выбора архитектуры

– Надёжность: допустима ли остановка при отказе управляющего узла

– Масштаб: сколько агентов сейчас и сколько через три года

– Скорость: характерные времена среды и допустимые задержки контура

– Степень координации: нужна жёсткая синхронизация или хватит согласованности

– Сложность, стоимость, безопасность и гибкость — ещё четыре группы факторов

Ключевая идея

Структурированная среда + надёжная связь + нужен оптимум → центр. Неструктурированная среда + плохая связь + нужна живучесть → рой. Большой масштаб + смешанные требования → иерархия.

7 Централизованный цикл управления

1 Сбор состояния

От каждого робота приходят позиция, статус, заряд

2 Матрица стоимостей

c[r][t] — оценка стоимости маршрута робота r к задаче t

3 Назначение

Венгерский алгоритм $O(n^3)$ либо жадное приближение arg min c[r][t]

4 Планирование

A* с резервированием пространственно-временных ячеек маршрута

5 Рассылка и контроль

Команды роботам; молчащий τ тактов помечается отказавшим

Шаг 4 возможен только потому, что центр видит все маршруты одновременно.

7 Децентрализованные алгоритмы

– Модель Рейнольдса: разделение, выравнивание, сплочение — три слагаемых

– Выравнивание скоростей — это в точности консенсус по векторам скорости

– Сложность $O(|N_i|) = O(1)$ на агента: масштабируется неограниченно

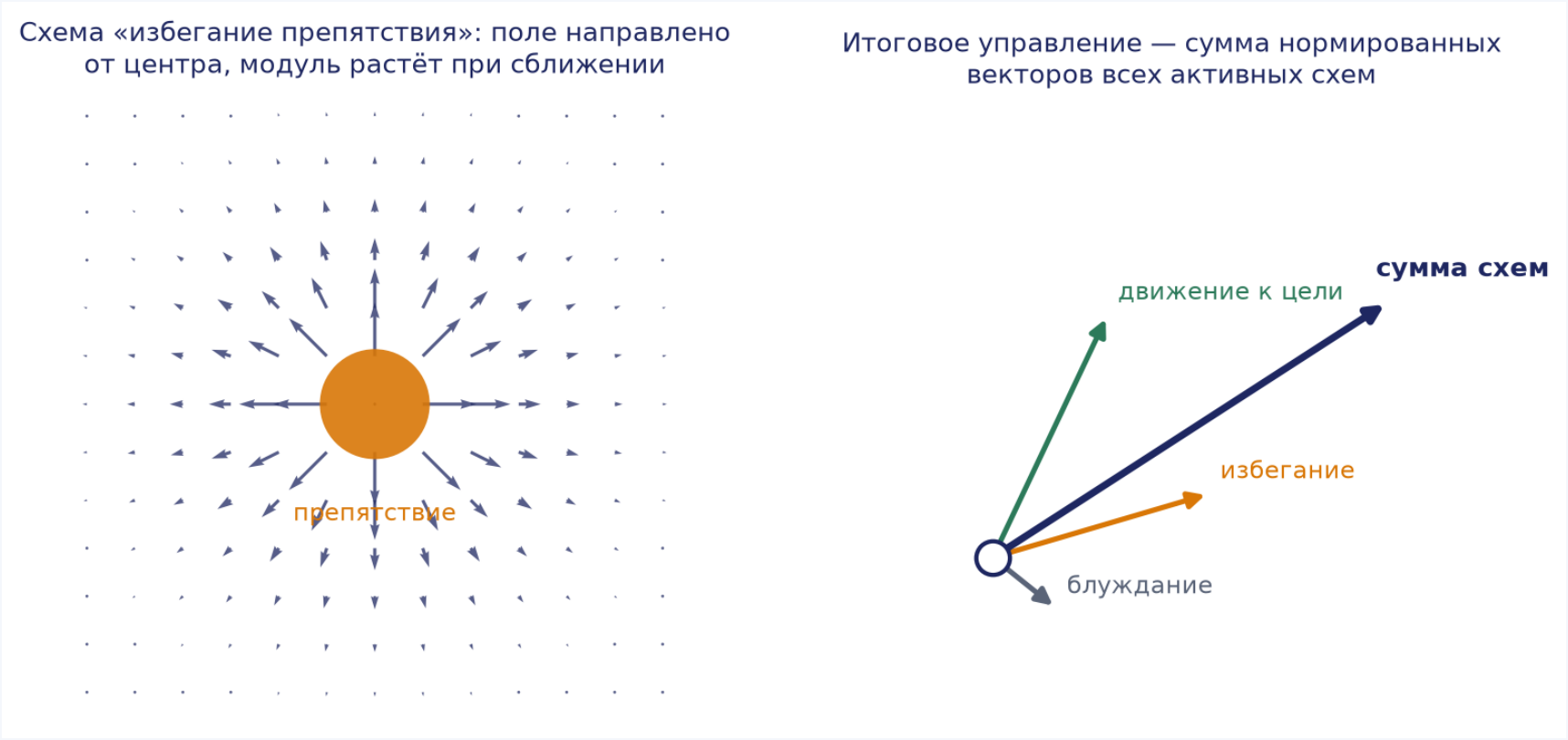
– Распределённые аукционы: агент объявляет торги соседям, задача уходит лучшей ставке

– CBBA гарантирует не менее 50 % от оптимума при субмодулярной полезности

Факт

Три правила Рейнольдса, придуманные в 1987 году для компьютерной анимации птичьих стай, до сих пор лежат в основе управления реальными роями БПЛА.

7 Моторные схемы Аркина: векторное поле



Поле избегания препятствия: вектор направлен радиально от центра препятствия, величина растёт при сближении. Итоговая команда — сумма нормированных векторов всех активных схем.

8 ROS 1 и ROS 2: цена мастера

ROS 1

- Обнаружение узлов через roscore
- Мастер — единая точка отказа
- Многороботность через multimaster
- Нет встроенных политик QoS
- Плохо дружит с ненадёжным радио

ROS 2

- Транспорт DDS, обнаружение распределённое
- Обмен peer-to-peer, мастера нет
- QoS: reliable/best effort, история, TTL
- SROS 2: аутентификация и шифрование
- Реальное время и жизненный цикл узлов

9 Три реальных кейса

Система	Архитектура	Почему так
Amazon Robotics (склад)	Централизованная, с зонированием	Структурированная среда, надёжная связь, экономика требует оптимума
Рой БПЛА мониторинга	Децентрализованная	Связь с землёй эпизодична, потери аппаратов ожидаемы
Автономный транспорт	Гибридная: облако + V2V + борт	Стратегия наверху, манёвры между соседями, торможение на борту

Закономерность: рост масштаба и неопределённости среды сдвигает архитектуру от центра к гибриду и рою.

10 Грабли реализации

– Задержки, джиттер и потери пакетов рассогласовывают картину мира агентов

– Лечение: устойчивые к задержкам протоколы, QoS, прогноз состояния соседей

– Безопасность: перехват команд, компрометация агента, глушение канала

– Эшелонированная защита: SROS 2, взаимная аутентификация, робастный консенсус

– Конфликты за ресурсы: тупики и голодание; лечатся приоритетами и RVO

Ключевая идея

Обязательное проектное требование — заранее заданное безопасное поведение при потере связи: остановиться, вернуться или автономно продолжить миссию. Молчание протокола — тоже решение, но плохое.

“

Проектный вопрос — не «центр или рой», а где именно провести границу между тем, что решает центр, и тем, что решает борт.

Итог лекции 3

Выводы

1. Архитектура задаётся графом G и распределением права решать между A и C
2. Централизация даёт оптимум, но упирается в вычисления быстрее, чем в канал
3. Единая точка отказа лечится резервированием ценой задачи выбора лидера
4. Децентрализация даёт O(1) на узел и живучесть ценой субоптимальности
5. Консенсус — базовый примитив; сходимость даёт связность, скорость — $\lambda_2(L)$
6. Иерархия и кластеризация при $m = \sqrt{n}$ снижают нагрузку худшего узла до $O(\sqrt{n})$
7. ROS 2 архитектурно нейтрален: DDS, QoS и SROS 2 поддержат любой из трёх вариантов

Вопросы для самопроверки

1. Что такое архитектура СУ РРТС и что означает каждый элемент кортежа $S = \langle A, C, G, П \rangle$?

2. Какова топология графа централизованной системы и нагрузка на центр как функция n?

3. Почему вычислительная сложность центра растёт быстрее линейной? Примеры $O(n^2)$ и $O(n^3)$

4. Что такое единая точка отказа? Что даёт и какие проблемы порождает резервирование?

5. Запишите протокол усредняющего консенсуса и условия его сходимости

6. Почему нагрузка на агента в рое равна O(1) и как это связано с масштабируемостью?

7. Что такое плавная деградация и κ(G)? Сколько византийцев выдержит сеть из N узлов?

8. Почему кластеризация с $m = \sqrt{n}$ балансирует нагрузку центра и координаторов?

9. Чем ROS 2 отличается от ROS 1 по механизму обнаружения узлов и почему это важно?

10. Для каждой из трёх архитектур приведите реальный пример и обоснуйте выбор